

Partie 2-Mise en place de contre-mesures

Mise en place d'une authentification par clés de chiffrement.....	1
Q1. Pourquoi l'algorithme ECDSA a été préféré à l'algorithme RSA lors de la génération de la paire de clés ?.....	2
Q2. À votre avis, pourquoi la mise en place de cette phrase de chiffrement pour accéder à la clé privée est extrêmement importante ?.....	2
Q3. Lister le contenu du répertoire \$HOME/.ssh/ puis afficher le contenu du fichier id_ecdsa.pub.....	2
Q4. Sur la machine cliente clissh.local.sio.fr, modifier les droits du fichier ~/.ssh/id_ecdsa (droits initiaux : 600 etusio:etusio) qui contient votre clé privée en 644. Se connecter sur le serveur distant srvssh.local.sio.fr qui contient la clé publique. Que se passe-t-il ? Pourquoi ?.....	3
Q5. Rétablir les droits de ~/.ssh/id_ecdsa sur le poste client. Maintenant sur le serveur distant, afficher les droits d'accès appliqués au fichier ~/.ssh/authorized_keys.....	3
Q6. Puis, modifier les droits de ~/.ssh/authorized_keys en 666. Se déconnecter puis se reconnecter. Vérifier si un changement est apparu ou non et tenter d'expliquer pourquoi (ne pas oublier de rétablir les droits d'origine ensuite).....	3
Q7. En analysant la connexion par clés, proposer une hypothèse de fonctionnement de cette nouvelle forme d'authentification. Expliquer ce qui différencie une connexion par mot de passe d'une connexion par clé de chiffrement.....	4
Utilisation de ssh-agent.....	4
Q8. Suite à la mise en place de cette authentification par clés de chiffrement, tenter à nouveau de simuler une attaque MITM entre le client et le serveur SSH. Quel résultat obtenez-vous ? Pourquoi ?.....	4
Faciliter la vérification de l'identité du serveur SSH avant la première connexion.....	5
Q9. Expliquer l'intérêt de cette démarche.....	5

Mise en place d'une authentification par clés de chiffrement

Éditer le fichier de configuration serveur et autoriser ce mécanisme de connexion :

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config
```

Redémarrer le service.

```
etusio@srvssh:~$ sudo systemctl restart sshd
```

Les manipulations sur le serveur sont maintenant terminées. Quitter éventuellement la connexion SSH sur le serveur puis basculer sur le client.

Sur la machine cliente, l'utilisateur etusio va devoir générer sa clé publique, et sa clé privée afin de pouvoir s'authentifier sur le serveur OpenSSH :

```
etusio@clissh:~ $ ssh-keygen -b 256 -t ecdsa
```

Q1. Pourquoi l'algorithme ECDSA a été préféré à l'algorithme RSA lors de la génération de la paire de clés ?

ECDSA est préféré car c'est un algorithme plus récent, plus compact et plus efficace, ce qui permet d'être plus rapide en prenant moins de place.

Q2. À votre avis, pourquoi la mise en place de cette phrase de chiffrement pour accéder à la clé privée est extrêmement importante ?

Cette phrase de chiffrement permet de protéger la clé privée de personnes malintentionnées, servant en fait de mot de passe.

Que faire de la paire de clés ? Si l'on récapitule, ssh-keygen a généré deux clés. Une clé privée qui est \$HOME/.ssh/id_ecdsa à laquelle vous seul devez avoir accès et une clé publique qui est \$HOME/.ssh/id_ecdsa.pub, qui peut être connue par tout le monde.

Q3. Lister le contenu du répertoire \$HOME/.ssh/ puis afficher le contenu du fichier id_ecdsa.pub.

On exécute la commande

```
ls -l /home/etusio/.ssh
```

```
etusio@srvssh:~$ ls -l /home/etusio/.ssh
total 8
-rw----- 1 etusio etusio 557 déc.  1 11:40 id_ecdsa
-rw-r--r-- 1 etusio etusio 175 déc.  1 11:40 id_ecdsa.pub
etusio@srvssh:~$
```

(pour moi le fichier s'était créé ici.)

ensuite on utilise

```
nano /home/etusio/.ssh/id_ecdsa.pub
```

```
GNU nano 3.2 /home/etusio/.ssh/id_ecdsa.pub
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoVTIubmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBFX8uvsIKBmZbyQEzdBI+CuU2CWvEGxkoBSLGHkcMog/zW2qphKqtF5UjqJl/z72JGKLYMWBsxeCruk557aVbo7 etusio@srvssh
```

Sur le serveur Debian, désactiver l'authentification par mot de passe pour juste conserver celle par clés dans le fichier de configuration /etc/ssh/sshd_config :

```
# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication no
#PermitEmptyPasswords no
```

Q4. Sur la machine cliente clissh.local.sio.fr, modifier les droits du fichier ~/.ssh/id_ecdsa (droits initiaux : 600 etusio:etusio) qui contient votre clé privée en 644. Se connecter sur le serveur distant srvssh.local.sio.fr qui contient la clé publique. Que se passe-t-il ? Pourquoi ?

La commande pour changer le mot de passe est:

```
chmod 664 /home/etusio/.ssh/id_ecdsa
```

Maintenant quand on essaye de se connecter on a ce message d'erreur.

```
etusio@clissh:~/.ssh$ ssh etusio@srvssh.local.sio.fr
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@                WARNING: UNPROTECTED PRIVATE KEY FILE!                @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0644 for '/home/etusio/.ssh/id_ecdsa' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "/home/etusio/.ssh/id_ecdsa": bad permissions
etusio@srvssh.local.sio.fr: Permission denied (publickey).
etusio@clissh:~/.ssh$
```

Q5. Rétablir les droits de ~/.ssh/id_ecdsa sur le poste client. Maintenant sur le serveur distant, afficher les droits d'accès appliqués au fichier ~/.ssh/authorized_keys.

```
etusio@clissh:~/.ssh$ chmod 600 id_ecdsa
```

On utilise la commande ls -l

```
etusio@srvssh:~$ ls -l /home/etusio/.ssh/authorized_keys
-rw----- 1 etusio etusio 175 déc.  2 08:24 /home/etusio/.ssh/authorized_keys
```

Q6. Puis, modifier les droits de ~/.ssh/authorized_keys en 666. Se déconnecter puis se reconnecter. Vérifier si un changement est apparu ou non et tenter d'expliquer pourquoi (ne pas oublier de rétablir les droits d'origine ensuite).

On change les droits.

```
etusio@srvssh:~$ cd /home/etusio/.ssh/
etusio@srvssh:~/.ssh$ chmod 666 authorized_keys
```

Et maintenant quand on essaie de se reconnecter:

```
etusio@clissh:~/.ssh$ ssh etusio@srvssh.local.sio.fr
etusio@srvssh.local.sio.fr: Permission denied (publickey).
```

On passe un fichier privé contenant la clé privée en public. C'est un autre moyen pour une personne malveillante de retrouver la clé privée si les droits sont configurés tel quel puisque n'importe qui peut lire cette clé

Q7. En analysant la connexion par clés, proposer une hypothèse de fonctionnement de cette nouvelle forme d'authentification. Expliquer ce qui différencie une connexion par mot de passe d'une connexion par clé de chiffrement.

Selon mon hypothèse, la connexion par clé fonctionne comme ceci:

Le serveur et le PC échangent une clé privée avec un algorithme puis, quand le pc essaie de se connecter, le serveur lui envoie une "phrase". Le PC utilise sa clé précédemment reliée pour modifier cette phrase et la renvoyer au serveur. Ce dernier n'a plus qu'à valider que le chiffrement est bon et l'échange de données peut se faire.

Utilisation de ssh-agent

Sur le client, taper les commandes suivantes, pour mettre en place le ssh-agent.

```
etusio@clissh:~/.ssh$ exec ssh-agent $SHELL
etusio@clissh:~/.ssh$ ssh-add
Enter passphrase for /home/etusio/.ssh/id_ecdsa:
Identity added: /home/etusio/.ssh/id_ecdsa (etusio@clissh)
```

Q8. Suite à la mise en place de cette authentification par clés de chiffrement, tenter à nouveau de simuler une attaque MITM entre le client et le serveur SSH. Quel résultat obtenez-vous ? Pourquoi ?

Lorsqu'on lance l'attaque dans la KALI avec **sudo ./start.sh** et **ettercap**

Quand on essaie de se connecter pendant l'attaque, on a désormais ce message d'erreur.

```
etusio@clissh:~/.ssh$ ssh etusio@srvssh.local.sio.fr
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:j7AWXna0OjldAG0uH9RTPyVuIfomHRhcU8s9FEoPcWE.
Please contact your system administrator.
Add correct host key in /home/etusio/.ssh/known_hosts to get rid of this message
.
Offending ECDSA key in /home/etusio/.ssh/known_hosts:2
  remove with:
  ssh-keygen -f "/home/etusio/.ssh/known_hosts" -R "srvssh.local.sio.fr"
ED25519 host key for srvssh.local.sio.fr has changed and you have requested stri
ct checking.
Host key verification failed.
etusio@clissh:~/.ssh$ █
```

La kali n'a pas appris la passphrase et l'agent n'accepte de connecter seulement ceux l'ayant déjà appris. Comme la connexion se fait automatiquement, la machine attaquante ne peut récupérer et/ou utiliser le mot de passe et le serveur nous bloque alors.

Faciliter la vérification de l'identité du serveur SSH avant la première connexion.

Le mécanisme SSHFP permet d'offrir plus de souplesse dans cette étape capitale de vérification

Tout d'abord, s'assurer que l'identité du serveur lors de la première connexion est bien celle de la clé publique présente sur le serveur

```
etusio@clissh:~/.ssh$ ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWlM4.
```

```
etusio@srvssh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
256 SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWlM4 root@srvssh (ECDSA)
etusio@srvssh:~$
```

Q9. Expliquer l'intérêt de cette démarche.

Lorsque le client établira une nouvelle connexion auprès d'un serveur SSH, il comparera l'empreinte de la clé publique proposée par le serveur avec celles enregistrées dans la zone DNS. Si elles sont identiques, alors l'identité du serveur est prouvée. Cela permet d'être plus sécurisé et plus simple à mettre en place.

Pour obtenir les enregistrements SSHFP à ajouter dans votre fichier de zone DNS, voici la commande à taper sur le serveur (C'est le nom d'hôte du serveur qui doit être saisi)

```
etusio@srvssh:~$ sudo ssh-keygen -r srvssh
[sudo] Mot de passe de etusio :
srvssh IN SSHFP 1 1 6fa8aeb8227f09bc4f8c8afe08245a8c7718d324
srvssh IN SSHFP 1 2 ef3c9989952d8591cc761f1b26a658354a93230dae730edbe42732bf0a1219a8
srvssh IN SSHFP 3 1 4c16f0e9298469630445a24c57c342b15540253f
srvssh IN SSHFP 3 2 94f23110ac6c6073cfde2ee23226595cb614a16f6f88bc127c5dfd30dal694ce
srvssh IN SSHFP 4 1 7ef6631af09e0585ff8c3169255189195f7b7ddf
srvssh IN SSHFP 4 2 88efa67bb697dafddelc18bee5c76218e8c78803bfa6b3e093c260cf56bd9f6e
```

Ajouter ces nouveaux enregistrements à la fin du fichier de zone présent sur le serveur SSH/DNS.

```
sudoedit /var/named/db.local.sio.fr
```

etusio@srvssh: ~

```
GNU nano 3.2

TTL 3600      ; 1 heure

local.sio.fr. IN SOA srvssh.local.sio.fr. postmaster.local.sio.fr. (
    2021030801      ; Serial
    1D              ; Refresh
    1H              ; Retry
    1W              ; Expire
    3H )            ; Negative Cache TTL

@      IN      NS      srvssh.local.sio.fr.

srvssh IN      A       192.168.56.10
clissh IN      A       192.168.56.11
rwbsd  IN      A       192.168.56.254

srvssh IN SSHFP 1 1 6fa8aeb8227f09bc4f8c8afe08245a8c7718d324
srvssh IN SSHFP 1 2 ef3c9989952d8591cc761flb26a658354a93230dae730edbe42732bf0a1219a8
srvssh IN SSHFP 3 1 4c16f0e9298469630445a24c57c342b15540253f
srvssh IN SSHFP 3 2 94f23110ac6c6073cfde2ee23226595cb614a16f6f88bc127c5dfd30dal694ce
srvssh IN SSHFP 4 1 7ef6631af09e0585ff8c3169255189195f7b7ddf
srvssh IN SSHFP 4 2 88efa67bb697dafddelc18bee5c76218e8c78803bfa6b3e093c260cf56bd9f6e
```

Recharger le service.

```
sudo systemctl reload bind9
```

Sur la machine cliente, si vous souhaitez que cette vérification se fasse au moment de la connexion, voici la commande à utiliser

```
ssh -o VerifyHostKeyDNS=true etusio@srvssh.local.sio.fr
```

Si vous souhaitez que cette vérification ait lieu à chaque nouvelle connexion depuis votre poste client, vous pouvez rajouter cette option dans le fichier de configuration du client (/etc/ssh/ssh_config) :

VerifyHostKeyDNS yes

Et on arrive à se connecter normalement si tout a été bien mis en place.